Sprint 10: State Management, Context API, External Libraries & React DevTools

සම්පූර්ණ සිංහල අධ්‍යයන සටහන (Comprehensive Sinhala Study Guide)

0. Where You Are Before Sprint 10 (Sprint 10 ආරම්භ කිරීමට පෙර ඔබ දැනගත යුතු දෑ)

- **Concept Explanation (Sinhala):** Sprint 10 පටන් ගන්න කලින් ඔයාට React වල තියෙන මේ මූලික සංකල්ප ගැන පැහැදිලි අවබෝධයක් තියෙන්න ඕනේ. මේවා කලින් Sprints වලදී අපි ඉගෙන ගත්තු දේවල්:
  - **React** කියන්නේ User Interfaces (UI) හදන්න පාවිච්චි කරන JavaScript library එකක්.
  - **Vite** කියන්නේ React projects ලේසියෙන්ම create කරලා run කරන්න උදව් වෙන tool එකක්.
  - **JSX** හරහා අපිට JavaScript ඇතුළේ HTML වගේ syntax ලියන්න පුළුවන් වෙනවා.
  - **Components** කියන්නේ අපිට නැවත නැවත පාවිච්චි කරන්න පුළුවන් UI කෑලි (reusable pieces of UI).
  - **Props** වලින් කරන්නේ parent component එකක ඉඳන් child component එකකට data pass කරන එක.
  - **children** prop එකෙන් පුළුවන් එක component එකකට තවත් JSX wrapper එකක් විදිහට වැඩ කරන්න.
  - **Conditional rendering** වලින් data වල ස්වභාවය අනුව වෙනස් UI එකක් පෙන්වන්න පුළුවන්.
  - List එකක් render කරන්න **map()** function එක පාවිච්චි කරනවා.
  - List එකක තියෙන items හරියට track කරන්න React වලට **stable keys** අවශ්‍ය වෙනවා.
  - Clicks, typing, සහ form submissions වලට React ප්‍රතිචාර දක්වන්නේ **Events** හරහා.
  - **Controlled inputs** මඟින් form වල values කෙලින්ම React state එකේ store කරගන්නවා.
  - වෙනස් වන data මතක තබා ගැනීමට **useState** පාවිච්චි කරනවා.
  - React වලදී state updates හැමවෙලාවෙම **immutable** වෙන්න ඕනේ (කෙලින්ම වෙනස් නොකර අලුත් copy එකක් විදිහට update කිරීම).
  - Object state එකක් update කරද්දී, එක field එකක් වෙනස් කරන්න කලින් මුළු object එකම copy කරගන්න ඕනේ.
  - Array state එකක් update කරන්න spread operator, **filter()**, සහ **map()** වගේ safe ක්‍රම පාවිච්චි කරන්න ඕනේ.
  - එකිනෙකට සම්බන්ධ state logic ලස්සනට organize කරන්න **useReducer** පාවිච්චි කරන්න පුළුවන්.
  - DOM elements වලට access ගන්න හෝ re-render එකක් සිදු නොකර අගයන් store කරගන්න **useRef** පාවිච්චි කරනවා.
  - React render එකකට පස්සේ සිද්ධ වෙන side effects run කරන්න **useEffect** පාවිච්චි කරනවා.
  - Old timers, listeners, සහ subscriptions නවත්තන්න **cleanup functions** ලියනවා.
  - Hook logic නැවත පාවිච්චි කරන්න **custom hooks** හදනවා.
  - App-level එකේ shared values (user, theme, language වගේ) share කරන්න **Context API** පාවිච්චි කරනවා.
  - APIs හෝ backend services එක්ක communicate කරලා data ගෙන්න ගන්න **data fetching** කරනවා.
  - Async UI එකක් හරියට පෙන්වන්න **loading**, **data**, සහ **error** states පාවිච්චි කරනවා.
  - අනවශ්‍ය re-renders වළක්වලා performance වැඩි කරන්න **useMemo**, **useCallback**, සහ **React.memo** උදව් වෙනවා.
  - URLs, React components එක්ක connect කරන්නේ **React Router** එකෙන්.
  - App එකට multi-page look එකක් දෙන්න dynamic routes, nested routes, 404 routes, protected routes, සහ lazy-loaded routes උදව් වෙනවා.

දැන් ඔයා screens කිහිපයක් තියෙන working React app එකක් හදන්න දන්නවා. Sprint 10 වලදී අපි අවධානය යොමු කරන්නේ වෙනස්ම ප්‍රශ්නයකට: **ලොකු React app එකක shared state එකක් හරියට organize කරන්නේ සහ debug කරන්නේ කොහොමද?**

1. Sprint 10 Goal (Sprint 10 හි ප්‍රධාන ඉලක්කය)

- **Concept Explanation (Sinhala):** ලොකු React apps වලදී state එක cleanly share කරගන්නේ කොහොමද සහ component tree එක විශාල වෙද්දී ඇතිවන ප්‍රශ්න debug කරගන්නේ කොහොමද කියලා ඉගෙන ගැනීම තමයි මේ sprint එකේ ප්‍රධාන ඉලක්කය. මෙම Sprint එක අවසානයේදී ඔබට:
  - Prop drilling කියන්නේ මොකක්ද සහ එය ලොකු apps වලදී ප්‍රශ්නයක් වෙන්නේ ඇයි කියලා පැහැදිලි කරන්න පුළුවන් වෙනවා.
  - Single source of truth සංකල්පය තේරුම් ගන්න ලැබෙනවා.
  - Local state එකක් සෑහෙන්නේ කවදාටද, state එකක් lift up කරන්න ඕනේ කවදාටද කියලා තීරණය කරන්න පුළුවන් වෙනවා.
  - Context API වැදගත් වෙන්නේ කවදාටද සහ එහි සීමාවන් මොනවාද කියලා දැනගන්න ලැබෙනවා.
  - Redux, Redux Toolkit, RTK Query, සහ Zustand වල කාර්යභාරය වටහා ගන්න පුළුවන් වෙනවා.
  - React DevTools පාවිච්චි කරලා props, state, context inspect කරන්නත්, Profiler tab එකෙන් re-renders ගැන හොයන්නත්, prop mismatch ප්‍රශ්න debug කරන්නත් ඉගෙන ගන්න ලැබෙනවා.

2. Big Mental Model (ප්‍රධාන මානසික ආකෘතිය)

- **Concept Explanation (Sinhala):** පොඩි React app එකකදී සාමාන්‍යයෙන් `useState` සහ `props` විතරක් පාවිච්චි කරලා local component state එකක් maintain කරන එක සෑහෙනවා. උදාහරණයක් විදිහට පහත counter component එක බලන්න. හැබැයි app එක ලොකු වෙද්දී අපිට shared data (logged-in user, theme, cart count, global notification, dashboard filters වගේ දේවල්) ගොඩක් components අතරේ share කරන්න වෙනවා. මේ වගේ වෙලාවට shared data වලට පැහැදිලි plan එකක් තියෙන්න ඕනේ.
  - **පොඩි app එකක් නම්:** Local state සහ props සෑහෙනවා.
  - **ලොකු app එකක් නම්:** Shared state එකට පැහැදිලි owner කෙනෙක් ඉන්න ඕනේ.
  - **ගොඩක් complex app එකක් නම්:** Context, Redux Toolkit, RTK Query, හෝ Zustand වගේ state management tool එකක් ඕනේ වෙනවා.
- **Code Block:**

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button type="button" onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

3. The Problem Sprint 10 Solves (Sprint 10 මඟින් විසඳන ප්‍රධාන ගැටලුව)

- **Concept Explanation (Sinhala):** හිතන්නකෝ අපේ app එකේ component tree එක මෙහෙමයි කියලා: `App -> Layout -> Header -> Navbar -> UserMenu` දැන් පහළම තියෙන `UserMenu` එකට දැනට ලොග් වෙලා ඉන්න user ගේ විස්තර (logged-in user) අවශ්‍ය වෙනවා. හැබැයි ඒ data එක තියෙන්නේ ඉහළම තියෙන `App` component එකේ. Context API පාවිච්චි කරන්නේ නැතුව මේ data එක යවන්න ගියොත්, අපිට `App` එකෙන් `Layout` එකටත්, `Layout` එකෙන් `Header` එකටත්, එතනින් `Navbar` එකටත්, අවසානයේ `UserMenu` එකටත් යවන්න වෙනවා. මෙතන තියෙන ප්‍රශ්නය තමයි `Layout`, `Header`, සහ `Navbar` කියන components තුනම මේ user data එක පාවිච්චි කරන්නේ නැහැ, හැබැයි පහළට pass කරන්න විතරක් උන් ඒවා props විදිහට ගන්න ඕනේ. මේකට තමයි **prop drilling** කියන්නේ.
- **Code Block:**

```
function App() {
  const user = { name: "Sangeeth" };
  return <Layout user={user} />;
}
```

4. Prop Drilling (Prop Drilling යනු කුමක්ද?)

4.1 What Is Prop Drilling?

- **Concept Explanation (Sinhala):** සරලවම කිව්වොත්, prop drilling කියන්නේ component tree එකේ මැද ඉන්න components වලට කිසිම වැඩක් නැති props, පහළම තියෙන component එකකට ලබා දීමේ අරමුණින් විතරක් උඩ ඉඳන් පහළට pass කිරීමයි. පහත කේතය ක්‍රියාත්මක වුණත්, app එක විශාල වෙද්දී මේක නඩත්තු කරන්න අමාරු වෙනවා. මැද ඉන්න components නිකන් පයිප්ප (pipes) වගේ විතරක් වැඩ කරන එක මඟහරවා ගන්න ඕනේ.
- **Code Block:**

```
function App() {
  const user = { name: "Sangeeth" };
  return <Layout user={user} />;
}

function Layout({ user }) {
  return <Header user={user} />;
}

function Header({ user }) {
  return <Navbar user={user} />;
}

function Navbar({ user }) {
  return <UserMenu user={user} />;
}

function UserMenu({ user }) {
  return <p>Logged in as {user.name}</p>;
}
```

4.2 Prop Drilling Mental Model

- **Concept Explanation (Sinhala):** `App (Data එක මෙහි තියෙනවා) ↓ (pass කරනවා) Layout (Data ලැබෙනවා, හැබැයි පාවිච්චි කරන්නේ නෑ) ↓ (pass කරනවා) Header (Data ලැබෙනවා, හැබැයි පාවිච්චි කරන්නේ නෑ) ↓ (pass කරනවා) Navbar (Data ලැබෙනවා, හැබැයි පාවිච්චි කරන්නේ නෑ) ↓ (pass කරනවා) UserMenu (අවසානයේ data එක පාවිච්චි කරනවා)` Component tree එක ගැඹුරු වෙන්න වෙන්න මේ වැඩේ කරන්න හරිම අමාරුයි.

4.3 Why Prop Drilling Becomes Hard (Prop Drilling අපහසු එකක් වෙන්නේ ඇයි?)

- **Concept Explanation (Sinhala):** Prop drilling නිසා මේ ප්‍රශ්න ඇති වෙනවා:
  - Levels වැඩි වෙද්දී නැවත නැවත props pass කරන්න ලියන්න වෙන කේතය (boilerplate) වැඩි වෙනවා.
  - Component එකක function parameters ගොඩක් වැඩි වෙලා අවුල් වෙනවා.
  - එක prop එකක නමක් වෙනස් කරන්න වුණොත්, ඒක pass වුණු හැම file එකකම නම වෙනස් කරන්න වෙනවා.
  - එක component එකක් වෙන තැනකට move කළොත් prop path එක බිඳ වැටෙන්න පුළුවන්.
  - Data එක කොහෙන්ද ආවේ කියලා trace කරන්න අමාරු වෙන නිසා debugging අපහසු වෙනවා.
- **Code Block:**

```
function Layout({ user, theme, cartCount, logout, toggleTheme }) {
  return (
    <Header
      user={user}
      theme={theme}
      cartCount={cartCount}
      logout={logout}
      toggleTheme={toggleTheme}
    />
  );
}
```

4.4 Is Prop Drilling Always Bad? (Prop Drilling හැමවෙලාවෙම නරකද?)

- **Concept Explanation (Sinhala):** නැහැ, සාමාන්‍ය ප්‍රමාණයට props pass කරන එක React වල සාමාන්‍ය දෙයක්. Parent කෙනෙක්ගෙන් Child කෙනෙක්ට (levels 1ක් හෝ 2ක්) කෙලින්ම යවන එක කිසිම ප්‍රශ්නයක් නැහැ. ප්‍රශ්නයක් වෙන්නේ data එක පාවිච්චි කරන්නේ නැති මැද තියෙන components ගොඩක් හරහා යවන්න සිද්ධ වුණොත් විතරයි.
- **Code Block:**

```
function App() {
  return <UserCard name="Sangeeth" />;
}

function UserCard({ name }) {
  return <h2>{name}</h2>;
}
```

5. Single Source of Truth (එක් විශ්වාසනීය තොරතුරු මූලාශ්‍රයක්)

5.1 What Is Single Source of Truth?

- **Concept Explanation (Sinhala):** සරලවම කිව්වොත්, shared data වලට හැමවෙලාවෙම එක පැහැදිලි owner කෙනෙක් විතරක් සිටිය යුතුයි යන්නයි. හැම component එකක්ම තමන්ට වෙන වෙනම data තියාගන්න ගියොත් (උදා: Navbar, Dashboard, Profile වල වෙන වෙනම user state තිබීම), UI එක එකිනෙකට වෙනස් වෙන්න පුළුවන් (inconsistent UI). එකම පොදු state එකක් තබාගෙන හැමෝම එතනින් read කරන එක තමයි නිවැරදි ක්‍රමය.

5.2 Why Single Source of Truth Matters (මෙය වැදගත් වෙන්නේ ඇයි?)

- **Concept Explanation (Sinhala):** - App එකේ ක්‍රියාකාරිත්වය තේරුම් ගන්න ලෙහෙසි වෙනවා.
  - Data එක තැනකදී වෙනස් වෙද්දී අනිත් තැන්වල පරණ data පෙන්වීම වළක්වනවා.
  - Updates සිද්ධ වෙන්නේ එකම තැනකදී පමණයි.
  - Debugging පහසු වෙනවා.

5.3 Where Can the Single Source of Truth Live? (මෙය තිබිය හැක්කේ කොහේද?)

- **Concept Explanation (Sinhala):** React වලදී අපිට මේ single source of truth එක විවිධ තැන්වල තියන්න පුළුවන්:
  - Local parent state එකක (useState)
  - Context Provider එකක
  - Redux store එකක
  - Zustand store එකක
  - Server cache / data-fetching store එකක

  මෙතනදී නිවැරදිම ක්‍රමය තෝරගන්නේ data එක කී දෙනෙක්ට අවශ්‍යද, කොච්චර නිතර වෙනස් වෙනවාද, සහ update rules කොච්චර සංකීර්ණද කියන කරුණු මතයි.

6. Local State, Lifted State, and Shared State (විවිධ අවස්ථාවලදී State තබා ගන්නා ආකාරය)

6.1 Local State

- **Concept Explanation (Sinhala):** එක component එකකට විතරක් යම් data එකක් අවශ්‍ය වෙනවා නම්, ඒක local state එකක් විදිහට තියාගන්න. උදාහරණයක් විදිහට search box එකක query එක තියාගන්න වෙනත් කිසිම කෙනෙක්ට ඒක ඕනේ නැත්නම් local state එකක් සෑහෙනවා.
- **Code Block:**

```
import { useState } from "react";

function SearchInput() {
  const [query, setQuery] = useState("");

  return (
    <input
      value={query}
      onChange={(event) => setQuery(event.target.value)}
      placeholder="Search..."
    />
  );
}
```

6.2 Lifting State Up (State එක ඉහළට එසවීම)

- **Concept Explanation (Sinhala):** එකම මට්ටමේ (sibling) තියෙන components දෙකකට එකම state එකක් අවශ්‍ය වුණොත්, ඒ state එක ඒ දෙන්නාගේම ළඟම තියෙන පොදු parent component එකට ගෙන යා යුතුයි (lifting state up).
- **Code Block:**

```
import { useState } from "react";

function SearchInput({ query, onQueryChange }) {
  return (
    <input
      value={query}
      onChange={(event) => onQueryChange(event.target.value)}
      placeholder="Search..."
    />
  );
}

function SearchPreview({ query }) {
  return <p>You searched for: {query}</p>;
}

export default function App() {
  const [query, setQuery] = useState("");

  return (
    <main>
      <h1>Search</h1>
      <SearchInput query={query} onQueryChange={setQuery} />
      <SearchPreview query={query} />
    </main>
  );
}
```

6.3 Shared App State

- **Concept Explanation (Sinhala):** හැබැයි data එක ඉතා ඈතින් පිහිටි components ගොඩකට එකවර අවශ්‍ය වෙනවා නම් (උදා: App -> Navbar, Dashboard, Settings, Profile හැම එකටම user ඕනේ නම්), state එක lift up කළත් prop drilling සිද්ධ වෙනවා. අන්න ඒ වගේ වෙලාවට තමයි **Context API** එක උදව් වෙන්නේ.

7. Context API (Context API යනු කුමක්ද?)

7.1 What Is Context API?

- **Concept Explanation (Sinhala):** Context API කියන්නේ React වලටම බිල්ට්-ඉන් (built-in) ලැබෙන tool එකක්. මේකෙන් පුළුවන් මැද තියෙන components වලට වදයක් නොවී කෙලින්ම ඕනෑම පහළ component එකකට data share කරන්න. වෙනම external packages install කරන්න අවශ්‍ය වෙන්නේ නැහැ. Theme, logged-in user, language, cart count වගේ දේවල් share කරන්න කියාපු tool එක.

7.2 What Problem Does Context Solve?

- **Concept Explanation (Sinhala):**
  - **Without Context:** `App -> Layout -> Header -> Navbar -> UserMenu` (Prop drilling)
  - **With Context:** `App -> AuthProvider` මඟින් data එක share කරන අතර, ඕනෑම component එකකට කෙලින්ම user data read කරන්න පුළුවන්.

7.3 Context API Has Three Main Pieces

- **Concept Explanation (Sinhala):** Context API එකක ප්‍රධාන කොටස් 3ක් තියෙනවා:
  1. **createContext():** Shared container එක හදනවා.
  2. **Provider:** Shared value එක හැමෝටම ලබා දෙනවා (share කරනවා).
  3. **useContext():** Shared value එක ඕනෑම child component එකකින් කියවනවා (read කරනවා).

7.4 createContext()

- **Concept Explanation (Sinhala):** Context object එකක් නිර්මාණය කරන්නේ මේ function එකෙන්. මෙහි ඇති `null` අගය default value එකයි.
- **Code Block:**

```
import { createContext } from "react";

const ThemeContext = createContext(null);
```

7.5 Provider

- **Concept Explanation (Sinhala):** Provider එක ඇතුළේ wrap කරලා තියෙන ඕනෑම component එකකට එහි `value` prop එකෙන් ලබා දෙන data කියවන්න පුළුවන් වෙනවා. Provider එකෙන් පිටත ඉන්න කෙනෙක්ට මේ context එක කියවන්න බැහැ.
- **Code Block:**

```
<ThemeContext.Provider value={value}>
  {children}
</ThemeContext.Provider>
```

7.6 useContext()

- **Concept Explanation (Sinhala):** මේ hook එක පාවිච්චි කරලා child components වලට පොදු container එකෙන් කෙලින්ම value එක read කරන්න පුළුවන්.
- **Code Block:**

```
import { useContext } from "react";

const themeData = useContext(ThemeContext);
```

8. Full Context Example: Theme Toggle (සම්පූර්ණ උදාහරණය: Theme Toggle)

- **Concept Explanation (Sinhala):** මේක තමයි Sprint 10 වලදී කරන්න තියෙන ප්‍රධානම ප්‍රායෝගික ව්‍යාපෘතිය (mini-project). මෙහිදී අපි `theme` එකක් හදලා, ඒක `Navbar` සහ `Home` අතරේ share කරනවා props pass කරන්නේ නැතුවම.

  Project Structure:
  - `src/context/ThemeContext.jsx` (Context එක හදන තැන)
  - `src/components/Navbar.jsx` (Theme එක toggle කරන button එක තියෙන තැන)
  - `src/pages/Home.jsx` (Theme එක පෙන්වන තැන)
  - `src/App.jsx` (Provider එකෙන් හැමදේම wrap කරන තැන)

8.1 File: `src/context/ThemeContext.jsx`

- **Code Block:**

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  function toggleTheme() {
    setTheme((currentTheme) =>
      currentTheme === "light" ? "dark" : "light"
    );
  }

  const value = {
    theme,
    toggleTheme,
  };

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  return useContext(ThemeContext);
}
```

8.2 Explanation of ThemeContext.jsx (ThemeContext.jsx පිළිබඳ සවිස්තර පැහැදිලි කිරීම)

- **Concept Explanation (Sinhala):**
  - **Import React tools:** `createContext` (context එක හදන්න), `useContext` (context එක read කරන්න), සහ `useState` (theme එක store කරන්න) import කරනවා.
  - **createContext(null):** `ThemeContext` නමින් පොදු shared container එකක් හදනවා.
  - **ThemeProvider({ children }):** මෙය wrapper component එකක්. මෙහි `children` කියන්නේ මේ Provider එක ඇතුළේ අපි දාන components (උදා: `Navbar`, `Home`).
  - **useState("light"):** Theme එක මුලින්ම "light" විදිහට තියාගන්නවා.
  - **toggleTheme():** දැනට තියෙන theme එක "light" නම් "dark" කරනවා, "dark" නම් "light" කරනවා. මෙහිදී current state එක මත අලුත් state එක තීරණය වන නිසා callback updater function එකක් පාවිච්චි කරලා තියෙනවා.
  - **const value:** Child components වලට access කරන්න ඕනේ දේවල් (theme, toggleTheme) එක object එකකට දානවා.
  - **ThemeContext.Provider:** මෙම value එක සියලුම children වෙත share කරනවා.
  - **useTheme():** හැම component එකකම `useContext(ThemeContext)` කියලා ලිය ලිය ඉන්නේ නැතුව කෙලින්ම `useTheme()` කියලා ලියන්න පුළුවන් වෙන්න හදපු ලස්සන custom hook එකක්.

8.3 File: `src/components/Navbar.jsx`

- **Code Block:**

```
import { useTheme } from "../context/ThemeContext";

export default function Navbar() {
  const { theme, toggleTheme } = useTheme();

  return (
    <nav>
      <p>Current theme: {theme}</p>
      <button type="button" onClick={toggleTheme}>
        Toggle theme
      </button>
    </nav>
  );
}
```

8.4 Explanation of Navbar.jsx

- **Concept Explanation (Sinhala):** `Navbar` component එක කිසිම prop එකක් parent ගෙන් ගන්නේ නැහැ. ඒ වෙනුවට අපේ custom hook එක වන `useTheme()` පාවිච්චි කරලා කෙලින්ම Context එකෙන් `theme` සහ `toggleTheme` ලබා ගන්නවා.

8.5 File: `src/pages/Home.jsx`

- **Code Block:**

```
import { useTheme } from "../context/ThemeContext";

export default function Home() {
  const { theme } = useTheme();

  return (
    <main>
      <h1>Home Page</h1>
      <p>This page is using the {theme} theme.</p>
    </main>
  );
}
```

8.6 Explanation of Home.jsx

- **Concept Explanation (Sinhala):** `Home` page එකත් කෙලින්ම `useTheme()` හරහා දැනට තියෙන theme එක කියවලා UI එක render කරනවා.

8.7 File: `src/App.jsx`

- **Code Block:**

```
import { ThemeProvider } from "./context/ThemeContext";
import Navbar from "./components/Navbar";
import Home from "./pages/Home";

export default function App() {
  return (
    <ThemeProvider>
      <Navbar />
      <Home />
    </ThemeProvider>
  );
}
```

8.8 Context Tree Mental Model

- **Concept Explanation (Sinhala):** `App` `└─ ThemeProvider` (Provider එක ඇතුළේ තමයි අනික් සියල්ල තියෙන්න ඕනේ) `├─ Navbar` (reads theme & toggleTheme) `└─ Home` (reads theme) **මතක තබාගත යුතු රීතිය:** Provider එකෙන් wrap කරපු කෙනෙක්ට විතරයි context value එක පාවිච්චි කරන්න පුළුවන්. `Navbar` එක `ThemeProvider` එකෙන් පිටත තිබ්බොත් වැඩ කරන්නේ නැහැ.

9. When Context API Is a Good Choice (Context API එක සුදුසුම අවස්ථා)

- **Concept Explanation (Sinhala):** App එක පුරාම විසිරිලා තියෙන ගොඩක් components වලට එකම data එකක් අවශ්‍ය වෙනවා නම් Context තමයි හොඳම විසඳුම.
  - Themes (Light/Dark mode)
  - Logged-in user information (Auth state)
  - Language settings (English/Sinhala/Tamil translation key)
  - Shopping cart count (හැම page එකකම උඩින් පෙන්වන්න ඕනේ නම්)
  - Global UI preferences (sidebar එක open ද close ද වගේ දේවල්)

10. When Context API Is Not a Good Choice (Context API එක නුසුදුසු අවස්ථා)

- **Concept Explanation (Sinhala):** Context එක හැම පුංචි state එකකටම පාවිච්චි කරන්න එපා.
  - හැම input field එකකම අගයන් Context දාන්න එපා.
  - Button එකක් hover වෙද්දී වෙනස් වෙන state එකක් Context දාන්න එපා.
  - තනි component එකකට විතරක් අදාළ modals හෝ පුංචි details Context දාන්න එපා.

  රීතිය: එක component එකකට විතරක් අදාළ නම් **local state** පාවිච්චි කරන්න. ළඟම ඉන්න sibling ට විතරක් ඕනේ නම් **lifting state up** කරන්න. ඈතින් ඉන්න ගොඩක් අයට ඕනේ නම් විතරක් **Context** පාවිච්චි කරන්න.

11. Context Warning: Unnecessary Re-renders (Context භාවිතයේදී ඇතිවන re-renders අවදානම)

- **Concept Explanation (Sinhala):** Context එකක් වෙනස් වෙද්දී, ඒ context එක පාවිච්චි කරන (useContext ලියලා තියෙන) හැම component එකක්ම re-render වෙනවා. මේක සාමාන්‍ය දෙයක් වුණත්, නිතර නිතර ඉතා වේගයෙන් වෙනස් වෙන values (උදා: හැම තත්පරේම වෙනස් වෙන timer එකක්, mouse position එක, input එකක type කරන හැම අකුරක්ම) Context එකකට දැම්මොත්, app එකේ තියෙන components සිය ගණනක් නිතර re-render වෙලා performance බහින්න පුළුවන්. ඒ නිසා නිතර වෙනස් නොවන stable values වලට විතරක් Context පාවිච්චි කරන්න.

12. Context API vs Props (Context සහ Props අතර වෙනස)

12.1 Use Props When Data Flow Is Simple

- **Concept Explanation (Sinhala):** Data එක කෙලින්ම parent ගෙන් child ට විතරක් යනවා නම් props පාවිච්චි කරන්න. ඒක සරලයි වගේම පිරිසිදුයි.
- **Code Block:**

```
function App() {
  return <UserCard name="Sangeeth" />;
}

function UserCard({ name }) {
  return <h2>{name}</h2>;
}
```

12.2 Use Context When Many Distant Components Need the Same Data

- **Concept Explanation (Sinhala):** දුරින් තියෙන components කිහිපයකටම එකම data එක ඕනේ නම් Context පාවිච්චි කරන්න.

12.3 Simple Decision Rule (සරල තීරණ ගැනීමේ නීතිය)

- එක child කෙනෙක්ට විතරක් ඕනේ නම්: **Props** පාවිච්චි කරන්න.
- Sibling components දෙකකට ඕනේ නම්: **Lift state up** කරන්න.
- දුරින් ඉන්න ගොඩක් අයට එකම data එක ඕනේ නම්: **Context API** පාවිච්චි කරන්න.

13. External State Management Libraries (බාහිර State Management පුස්තකාල)

- **Concept Explanation (Sinhala):** Context API එක හැමවෙලේම හරියන්නේ නැති නිසා ලොකු, සකීර්ණ apps හදද්දී අපිට බාහිර libraries පාවිච්චි කරන්න වෙනවා. Sprint 10 වලදී අපිට Redux, Redux Toolkit, RTK Query, සහ Zustand හඳුන්වා දෙනවා. මේවා දැනටම ප්‍රගුණ කරන්න අවශ්‍ය නැහැ, ඒවා තියෙන්නේ ඇයි කියලා තේරුම් ගත්තාම සෑහෙනවා.

14. Redux (Redux යනු කුමක්ද?)

14.1 What Is Redux?

- **Concept Explanation (Sinhala):** Redux කියන්නේ මුළු app එකේම තියෙන shared state එක එකම පොදු "central store" එකක තියාගන්න පාවිච්චි කරන library එකක්. App එකේ state එක වෙනස් වන ආකාරය ඉතාම පැහැදිලිව සහ Predictable (කලින්ම කිව හැකි) මට්ටමක පවත්වා ගන්න මේක උදව් වෙනවා.

14.2 Redux Mental Model

- **Concept Explanation (Sinhala):** `Component (dispatch කරනවා) ↓ Action (මොකක්ද වුණේ කියලා කියන message එක) ↓ Reducer (පරණ state එකයි action එකයි අරන් අලුත් state එක හදනවා) ↓ Store (අලුත් state එක save කරගන්නවා) ↓ UI එක update වෙනවා`

14.3 Important Redux Terms

- **Concept Explanation (Sinhala):**
  - **store:** මුළු app එකේම state එක එකට තියෙන මධ්‍යම බඳුන (central container).
  - **action:** මොකක්ද සිද්ධ වුණේ කියලා විස්තර කරන JavaScript object එකක් (උදා: cart එකට item එකක් එකතු වුණා).
  - **reducer:** පරණ state එකයි action එකයි අරගෙන අලුත් state එක calculate කරන පිරිසිදු function එකක්.
  - **dispatch:** Action එකක් store එකට යවන්න (send කරන්න) පාවිච්චි කරන ක්‍රමය.

14.4 Store

- **Concept Explanation (Sinhala):** මුළු app එකේම shared state එක එකම තැනක තියෙන්නේ මෙතනයි. (User details, Cart items, Notification, Permissions ආදී සියල්ල).

14.5 Action

- **Concept Explanation (Sinhala):** සිදු වුණු දේ විස්තර කරන message එකක්.
- **Code Block:**

```
{
  type: "cart/addItem",
  payload: {
    id: "p1",
    name: "Keyboard"
  }
}
```

14.6 Reducer

- **Concept Explanation (Sinhala):** පරණ state එක කෙලින්ම mutate (වෙනස්) කරන්නේ නැතුව අලුත් state එකක් හදලා දෙන function එකක්. (current state + action -> new state).

14.7 Dispatch

- **Concept Explanation (Sinhala):** Redux එකට යම් දෙයක් සිද්ධ වුණා කියලා දැනුම් දෙන්න dispatch පාවිච්චි කරනවා.
- **Code Block:**

```
dispatch({ type: "cart/addItem", payload: product });
```

14.8 When Redux Is Useful

- **Concept Explanation (Sinhala):** - ඉතා විශාල components ප්‍රමාණයකට එකම state එකක් ඕනේ නම්.
  - State update වන ආකාරය ගොඩක් සංකීර්ණ නම්.
  - විශාල team එකක් එකට වැඩ කරනවා නම්.
  - App එකේ සංකීර්ණ business logic තියෙනවා නම්.

14.9 When Redux May Be Too Much

- **Concept Explanation (Sinhala):** App එක පොඩි නම්, Context එකෙන්ම වැඩේ කරගන්න පුළුවන් නම්, සහ නිකන්ම "professional" පෙනුමක් ගන්න විතරක් Redux පාවිච්චි කරන්න හිතනවා නම් Redux දාන්න එපා. ඒකෙන් කේතය අනවශ්‍ය විදිහට සංකීර්ණ වෙනවා.

15. Redux Toolkit (Redux Toolkit යනු කුමක්ද?)

15.1 What Is Redux Toolkit?

- **Concept Explanation (Sinhala):** Redux Toolkit (RTK) කියන්නේ පරණ සාම්ප්‍රදායික Redux ලියන එක පහසු කරන්න හදපු නිල, නවීන ක්‍රමයයි. මේකෙන් Redux setup එක ඉතාම සරල කරලා දෙනවා.

15.2 Why Redux Toolkit Exists

- **Concept Explanation (Sinhala):** පරණ ක්‍රමයට Redux ලියද්දී එක පොඩි දේකටත් files කිහිපයක් හදලා විශාල කේත ප්‍රමාණයක් (boilerplate) ලියන්න වුණා. Redux Toolkit එකෙන් ඒ හැමදේම ලස්සනට සරල කරලා දෙනවා.

15.3 What Redux Toolkit Helps With

- **Concept Explanation (Sinhala):** Stores සෑදීම, reducers සහ actions ලේසියෙන් සෑදීම, සහ state slices ලස්සනට organize කරන්න උදව් වෙනවා.

15.4 Beginner Rule

- **Concept Explanation (Sinhala):** ඔයා නවීන React app එකකට Redux පාවිච්චි කරන්න හිතනවා නම්, පරණ ක්‍රමය ඉගෙන ගන්නේ නැතුව කෙලින්ම **Redux Toolkit (RTK)** ඉගෙන ගන්න.

16. RTK Query (RTK Query යනු කුමක්ද?)

16.1 What Is RTK Query?

- **Concept Explanation (Sinhala):** RTK Query කියන්නේ Redux Toolkit එකේම කොටසක්. මේක විශේෂයෙන්ම හදලා තියෙන්නේ **API/Server data** (backend එකෙන් ගන්නා data) fetch කරන්න, cache කරන්න, සහ refetch කරන්නයි.

16.2 What Problem Does RTK Query Solve?

- **Concept Explanation (Sinhala):** කලින් sprints වලදී අපි `loading`, `data`, `error` states අතින් ලියලා data fetch කළා. හැබැයි ලොකු app එකකදී එකම API call එක තැන් කිහිපයක කරන්න වුණොත්, manual caching, automatic refetching (data අලුත් කිරීම) වගේ දේවල් අතින් කරන්න ගියාම පිස්සු හැදෙනවා. RTK Query එකෙන් ඒ හැම ප්‍රශ්නයක්ම නිකන්ම විසඳලා දෙනවා.
- **Code Block:**

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

16.3 RTK Query Helps With

- **Concept Explanation (Sinhala):** Data fetch කිරීම, automatic loading/error states හැසිරවීම, caching (දැනටමත් ගෙනාපු data නැවත නැවත backend එකෙන් නොඉල්ලීම), සහ API data පිළිවෙළකට තබා ගැනීම.

16.4 When RTK Query Is Useful

- **Concept Explanation (Sinhala):** App එකේ ගොඩක් API requests තියෙනවා නම්, data caching අවශ්‍ය නම්, සහ දැනටමත් Redux Toolkit පාවිච්චි කරනවා නම් RTK Query නියාපු බහක්ඩය.

16.5 Client State vs Server State

- **Concept Explanation (Sinhala):**
  - **Client State:** Frontend එක ඇතුළේ විතරක් තියෙන දේවල් (Theme, modal open/close, sidebar expanded/collapsed). මේවා Redux/Zustand/Context වලින් ලේසියෙන් පාලනය කරන්න පුළුවන්.
  - **Server State:** Database එකක තියෙන, API එකක් හරහා ගන්නා data (Products list, user profile, orders). මේවා පාලනය කරන්න RTK Query වගේ tool එකක් සුදුසුයි.

17. Zustand (Zustand යනු කුමක්ද?)

17.1 What Is Zustand?

- **Concept Explanation (Sinhala):** Zustand කියන්නේ Redux වලට වඩා ඉතාම සැහැල්ලු (lightweight), සරල state management library එකක්. මේක ජර්මන් භාෂාවෙන් "Zustand" කියන්නේ "State" කියන එකටයි.

17.2 Zustand Mental Model

- **Concept Explanation (Sinhala):** 1. Store එකක් හදනවා. 2. ඒක ඇතුළේ shared state එකයි, ඒක update කරන functions ටියි දෙකම දානවා. 3. ඕනෑම component එකක් ඒ hook එක පාවිච්චි කරලා කෙලින්ම store එකෙන් data කියවනවා හෝ update කරනවා.

17.3 Why Developers Like Zustand (Developerලා මේකට ආදරේ ඇයි?)

- **Concept Explanation (Sinhala):** Redux වල වගේ අනවශ්‍ය boilerplate (වැල් වටාරම් කේත) ලියන්න ඕනේ නැහැ, හරිම සරලයි. App එක Provider එකකින් wrap කරන්නවත් අවශ්‍ය වෙන්නේ නැහැ. Hooks විදිහටම වැඩ කරන්න පුළුවන්.

17.4 When Zustand Is Useful

- **Concept Explanation (Sinhala):** පොඩි හෝ මධ්‍යම ප්‍රමාණයේ apps වලට, global UI state එකක් තියාගන්න, සහ Context වලට වඩා ටිකක් ලොකු හැබැයි Redux තරම් සංකීර්ණ නැති වැඩ වලට Zustand නියාපු බඩු.

17.5 Zustand vs Redux Toolkit

- **Concept Explanation (Sinhala):** - **Redux Toolkit:** ගොඩක් structured, ලොකු apps වලට සහ predictable state flow එකක් ඕනේ වෙලාවට සුදුසුයි.
  - **Zustand:** වඩාත් සැහැල්ලු සහ සරලයි. අඩු ලැහැස්ති වීමකින් (less setup) ඉක්මනින් වැඩක් කරගන්න සුදුසුයි.

18. Comparison & Decision Tables (සංසන්දනාත්මක සටහන්)

18.1 Comparison Table

- **Concept Explanation (Sinhala):**

Tool	Main Use	Best For	Beginner Warning
Local state	Component-owned data	One component	Do not lift too early
Lifted state	Shared sibling data	Nearby components	Can become prop drilling if overused
Context API	Simple shared app values	Theme, auth user, language	Not for every tiny state value
Redux Toolkit	Complex global client state	Large apps, predictable updates	More setup than Context
RTK Query	Server/API state	Fetching, caching, refetching	Learn basic fetch first
Zustand	Lightweight global state	Small/medium shared state	Still an external dependency

18.2 Beginner Decision Table

- **Concept Explanation (Sinhala):**

Situation	Use
එක component එකකට විතරක් data ඕනේ නම්	`useState`
ළඟම ඉන්න sibling component දෙකකට data ඕනේ නම්	Lift state up
Parent කෙනෙක් කෙලින්ම child කෙනෙක්ට data යවද්දී	Props
දුරින් ඉන්න ගොඩක් අයට සරල shared data ඕනේ නම්	Context API
සංකීර්ණ update rules එක්ක ලොකු global state එකක් ඕනේ නම්	Redux Toolkit
API/server data caching එක්ක ඕනේ නම්	RTK Query
සරල global store එකක් ලේසියෙන් ඕනේ නම්	Zustand

18.3 Default Beginner Order (ආරම්භකයකු යා යුතු නිවැරදි පිළිවෙළ)

- **Concept Explanation (Sinhala):** `useState` —> `lift state up` —> `props` —> `Context` —> `external state library (Zustand/Redux)` කිසිම වෙලාවක app එකක් පටන් ගත්තු ගමන්ම කෙලින්ම Redux හෝ Zustand පාවිච්චි කරන්න එපා. සරලම ක්‍රමයෙන් පටන් අරන් අවශ්‍යතාවය අනුව ඉහළට යන්න.

19. React DevTools (React DevTools යනු කුමක්ද?)

19.1 What Is React DevTools?

- **Concept Explanation (Sinhala):** React DevTools කියන්නේ browser extension එකක්. මේකෙන් අපිට React app එකක් ඇතුළේ සිද්ධ වෙන දේවල් (Component tree, props, state, context, rendering behavior) ඇස් දෙකෙන්ම බලාගන්න පුළුවන්. අනුමාන කර කර ඉන්නේ නැතුව හරියටම ප්‍රශ්නය debug කරන්න මේක අත්‍යවශ්‍යයි.

19.2 Main React DevTools Tabs

- **Concept Explanation (Sinhala):** මෙහි ප්‍රධාන වශයෙන් tabs දෙකක් තියෙනවා:
  1. **Components tab**
  2. **Profiler tab**

20. Components Tab (Components Tab එක)

20.1 What Is the Components Tab?

- **Concept Explanation (Sinhala):** Components Tab එකෙන් ඔයාට screen එකේ තියෙන මුළු React component tree එකම බලාගන්න පුළුවන්. ඒ වගේම:
  - Screen එකේ තියෙන්නේ මොන components ද?
  - තෝරාගත් component එකට ලැබිලා තියෙන props මොනවාද?
  - ඒකේ දැනට තියෙන state values මොනවාද?
  - ඒක කියවන Context value එක මොකක්ද?
  - ඒකේ parent කවුද, children කවුද? ආදී සියලුම දේ බලාගන්න පුළුවන්.

20.2 Why the Components Tab Is Useful

- **Concept Explanation (Sinhala):** UI එකේ මොකක් හරි වැරැද්දක් පෙන්වද්දී, "මම යවපු prop එක හරියට ගියාද?", "State එක update වුණාද?", "Component එක තියෙන්නේ Provider එක ඇතුළේද?" වගේ ප්‍රශ්න වලට හරියටම උත්තර හොයාගන්න මේක උදව් වෙනවා.

20.3 Example: Inspecting Props

- **Concept Explanation (Sinhala):** React DevTools වල Child component එක select කරලා බැලුවොත්, දකුණු පැත්තේ panel එකෙන් `message: "Hello"` කියලා වැටිලා තියෙනවා දකින්න පුළුවන්. Prop එක වැටිලා නැත්නම් parent එකෙන් හරියට pass කරලා නැහැ.
- **Code Block:**

```
function Parent() {
  return <Child message="Hello" />;
}

function Child({ message }) {
  return <p>{message}</p>;
}
```

20.4 Example: Inspecting State

- **Concept Explanation (Sinhala):** React DevTools වල Counter select කරලා බැලුවොත්, එහි දැනට තියෙන state එක බලාගන්න පුළුවන්. Button එක click කරද්දී state එක වෙනස් වෙනවද කියලා බලාගන්නත් පුළුවන්.
- **Code Block:**

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button type="button" onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

20.5 Example: Inspecting Context

- **Concept Explanation (Sinhala):** Component එකක් Context එකක් පාවිච්චි කරද්දී, එයට නිවැරදි value එක ලැබෙනවාද, එය Provider එක ඇතුළේද තියෙන්නේ කියලා Components Tab එකෙන් ලේසියෙන්ම බලාගන්න පුළුවන්.

21. Profiler Tab (Profiler Tab එක)

21.1 What Is the Profiler Tab?

- **Concept Explanation (Sinhala):** Profiler Tab එකෙන් කරන්නේ app එකේ rendering performance මනින එකයි. "ඇයි මේ component එක re-render වුණේ?", "Re-render වෙන්න කොච්චර වෙලාවක් ගියාද?", "Context එකක් නිසා අනවශ්‍ය අය Render වුණාද?" වගේ ප්‍රශ්න වලට උත්තර හොයාගන්න මේක පාවිච්චි කරනවා.

21.2 Beginner Profiler Workflow

- **Concept Explanation (Sinhala):**
  1. React DevTools වල Profiler tab එකට යන්න.
  2. Record button එක (නිල් පාට රවුම) click කරන්න.
  3. App එකේ clicks කරන්න (theme එක toggle කරන්න වගේ දේවල්).
  4. Record එක stop කරන්න.
  5. Render වුණු components දිහා බලලා, අනවශ්‍ය re-renders සිද්ධ වෙලාද කියලා පරීක්ෂා කරන්න.

21.3 What Is a Re-render?

- **Concept Explanation (Sinhala):** React විසින් component function එක නැවත වරක් run කරලා අලුත් UI එක ගණනය කිරීමට re-render එකක් කියනවා. Re-render එකක් වෙන්නේ:
  - State වෙනස් වුණොත්
  - Props වෙනස් වුණොත්
  - Parent component එකක් re-render වුණොත්
  - පාවිච්චි කරන Context value එකක් වෙනස් වුණොත්

  Re-render වෙන එක නරක දෙයක් නෙමෙයි, හැබැයි කිසිම වැඩක් නැතුව අනවශ්‍ය විදිහට ලොකු components නිතර re-render වෙන එක තමයි performance බස්සන ප්‍රශ්නය.

21.4 Context and Re-renders

- **Concept Explanation (Sinhala):** Theme එක toggle කරද්දී Navbar, Home, SettingsPanel වගේ හැමෝම re-render වෙන එක සාමාන්‍ය දෙයක් මොකද ඒ හැමෝම theme එක පාවිච්චි කරනවා. හැබැයි කිසිම සම්බන්ධයක් නැති components රෙන්ඩර් වෙනවා නම් ඔයාගේ context structure එක ගැන ආයෙත් හිතන්න වෙනවා.

22. Prop Mismatch Debugging (Prop Mismatch Debugging කරන ආකාරය)

22.1 What Is a Prop Mismatch?

- **Concept Explanation (Sinhala):** Prop mismatch එකක් කියන්නේ Parent component එකෙන් යවන prop එකේ නමයි, Child component එකෙන් බලාපොරොත්තු වෙන prop එකේ නමයි දෙකක් වීමයි. පහත වැරදි කේතය (broken code) බලන්න:
  - `Child` එකෙන් pass කරන්නේ `reply` කියන නම.
  - හැබැයි `Grandchild` එකෙන් බලාපොරොත්තු වෙන්නේ `response` කියන නම.
  - ඒ නිසා `Grandchild` එකට `undefined` ලැබෙනවා.
- **Code Block:**

```
// BROKEN CODE
function Parent() {
  return <Child response="Hello" />;
}

function Child({ response }) {
  return <Grandchild reply={response} />;
}

function Grandchild({ response }) {
  return <h2>{response}</h2>;
}
```

22.2 Fixed Version 1

- **Concept Explanation (Sinhala):** මේක විසඳන්න පුළුවන් එක ක්‍රමයක් තමයි parent එකෙන් pass කරන prop එකේ නමයි, child එකෙන් ගන්න නමයි දෙකම එකම නමක් (`response`) බවට පත් කිරීම.
- **Code Block:**

```
function Parent() {
  return <Child response="Hello" />;
}

function Child({ response }) {
  return <Grandchild response={response} />;
}

function Grandchild({ response }) {
  return <h2>{response}</h2>;
}
```

22.3 Fixed Version 2

- **Concept Explanation (Sinhala):** නැත්නම් child එක බලාපොරොත්තු වෙන නම `reply` විදිහට වෙනස් කරන්නත් පුළුවන්. ක්‍රම දෙකම නිවැරදියි. වැදගත්ම රීතිය තමයි: **Parent යවන නමමයි Child බලාපොරොත්තු වෙන්න ඕනේ.**
- **Code Block:**

```
function Parent() {
  return <Child response="Hello" />;
}

function Child({ response }) {
  return <Grandchild reply={response} />;
}

function Grandchild({ reply }) {
  return <h2>{reply}</h2>;
}
```

22.4 How React DevTools Helps With Prop Mismatch

- **Concept Explanation (Sinhala):** UI එකේ යම් අගයක් පෙන්වන්නේ නැත්නම්:
  1. React DevTools open කරන්න.
  2. ඒ component එක select කරන්න.
  3. Props පරීක්ෂා කරන්න.
  4. Prop එක undefined ද කියලා බලන්න.
  5. Parent component එකට ගිහින් යවපු නමයි, මුන් බලාපොරොත්තු වෙන නමයි සමානද කියලා බලලා fix කරන්න.

23. Sprint 10 Mini Project: Theme Toggle With Context (Mini Project එකේ ඉලක්කය)

- **Concept Explanation (Sinhala):** අපිට හදන්න තියෙන component tree එක මෙන්න මේකයි: `App -> ThemeProvider -> [Navbar, HomePage, SettingsPanel]`

  ලක්ෂණ:
  - `Navbar` එකේ current theme එක පෙන්වනවා වගේම Toggle button එකක් තියෙනවා.
  - `HomePage` එක සහ `SettingsPanel` එකත් theme එක කියවනවා.
  - කිසිම තැනක props drilling නැහැ.
  - Click කරද්දී theme එක වෙනස් වෙලා ඔක්කොම update වෙනවා.

24. Common Beginner Mistakes (ආධුනිකයන් සිදුකරන පොදු වැරදි)

- **Mistake 1: Using Context for Everything**
  - **Sinhala:** හැම පොඩි input state එකක්ම Context දාන්න යාම වැරදියි. Local state එක හැමවෙලාවෙම local තියන්න.
- **Mistake 2: Forgetting the Provider**
  - **Sinhala:** Context එකක් පාවිච්චි කරන component එකක් `ThemeProvider` එකෙන් පිටත තිබ්බොත් වැඩ කරන්නේ නැහැ. අනිවාර්යයෙන්ම wrap කරන්න ඕනේ.
- **Mistake 3: Prop Name Mismatch**
  - **Sinhala:** Parent එකෙන් `reply` කියලා යවලා child එකෙන් `response` කියලා ගන්න ගියාම වැරදීම.
- **Mistake 4: Jumping to Redux Too Early**
  - **Sinhala:** පොඩි app එකකටත් Redux දාන්න ගිහින් කේතය සංකීර්ණ කරගැනීම.
- **Mistake 5: Ignoring React DevTools**
  - **Sinhala:** Console එකේ error එකක් ආවාම හෝ UI එක වැරදියට පෙන්වද්දී DevTools පාවිච්චි නොකර අනුමාන කර කර ඉන්න එක.

25. Sprint 10 Debug Task (ප්‍රායෝගික Debugging කේතය)

- **Concept Explanation (Sinhala):** පහත තියෙන්නේ broken code එකක්. `Child` එකෙන් `reply` කියලා යවද්දී `Grandchild` එකෙන් `response` බලාපොරොත්තු වෙන නිසා ප්‍රශ්නයක් ඇති වෙනවා. එය නිවැරදි කර ඇති ආකාර 2ක් පහත දැක්වේ.
- **Code Block:**

```
// BROKEN CODE
function Child({ response }) {
  return <Grandchild reply={response} />;
}

function Grandchild({ response }) {
  return <h2>{response}</h2>;
}

// FIXED CODE OPTION 1
function Child({ response }) {
  return <Grandchild response={response} />;
}

function Grandchild({ response }) {
  return <h2>{response}</h2>;
}

// FIXED CODE OPTION 2
function Child({ response }) {
  return <Grandchild reply={response} />;
}

function Grandchild({ reply }) {
  return <h2>{reply}</h2>;
}
```

26. Sprint 10 Revision Checkpoint Questions (ප්‍රශ්නෝත්තර සටහන)

- **Concept Explanation (Sinhala):** මේ ප්‍රශ්න වලට උත්තර ඔයාට තනියම හිතන්න පුළුවන්ද කියලා බලන්න (විභාගයට වැදගත්):
  1. **What is prop drilling?** (මැද ඉන්න අයට අනවශ්‍ය props පහළට pass කිරීම)
  2. **Why does prop drilling become hard to maintain?** (කේතය වැඩි වීම, නම වෙනස් කිරීමේ අපහසුතාව)
  3. **Is all prop passing bad?** (නැත, 1-2 levels pass කිරීම සාමාන්‍යයි)
  4. **What does "single source of truth" mean?** (Shared data වලට එක පැහැදිලි owner කෙනෙක් සිටීම)
  5. **Why is single source of truth useful?** (UI inconsistency වළක්වයි, debugging පහසු කරයි)
  6. **When should state stay local?** (තනි component එකකට විතරක් ඕනේ නම්)
  7. **When should you lift state up?** (Sibling components දෙකකට පොදු state එකක් ඕනේ නම්)
  8. **When is Context better than normal props?** (ඈතින් පිහිටි ගොඩක් අයට එකම data ඕනේ නම්)
  9. **What are the three main pieces of Context API?** (createContext, Provider, useContext)
  10. **What does createContext() do?** (Shared container එකක් සාදයි)
  11. **What does the Provider do?** (Value එක children වෙත share කරයි)
  12. **What does useContext() do?** (Child component එකකදී shared value එක read කරයි)
  13. **Why is a custom context hook like useTheme() useful?** (කේතය පිරිසිදු කරයි)
  14. **What kind of values are good for Context?** (Theme, User, Language වැනි stable values)
  15. **What kind of values should usually not go into Context?** (නිතර වේගයෙන් වෙනස් වන input values, timers)
  16. **Why can Context cause unnecessary re-renders?** (Value එකක් වෙනස් වෙද්දී එය කියවන හැමෝම render වන නිසා)
  17. **What problem does Redux solve?** (සංකීර්ණ global state, predictable updates)
  18. **What is a Redux store?** (Central state container එක)
  19. **What is a Redux action?** (සිදු වූ දේ විස්තර කරන message එක)
  20. **What is a Redux reducer?** (අලුත් state එක ගණනය කරන function එක)
  21. **What does dispatch mean?** (Action එකක් store එකට send කිරීම)
  22. **Why does Redux Toolkit exist?** (පරණ Redux වල කේතය ලියන එක සරල කිරීමට)
  23. **What problem does RTK Query solve?** (API data loading, error, caching, refetching ලෙහෙසි කරයි)
  24. **What kind of state is server state?** (Backend/Database එකෙන් එන data)
  25. **What kind of state is client state?** (Frontend එක ඇතුළේ තියෙන UI states)
  26. **What makes Zustand lighter than Redux?** (බොයිලර්ප්ලේට් අඩුයි, provider එකක් නැතත් වැඩ කළ හැක)
  27. **Which React DevTools tab shows props, state, and context?** (Components Tab)
  28. **Which React DevTools tab helps investigate re-renders?** (Profiler Tab)
  29. **What is a prop mismatch?** (Parent යවන prop නමයි Child බලාපොරොත්තු වන නමයි වෙනස් වීම)
  30. **How can React DevTools help debug a prop mismatch?** (Components Tab එකෙන් props inspect කිරීමෙන්)

27. Practice Tasks (ප්‍රායෝගික අභ්‍යාස)

Task 5: Fix a Prop Mismatch

- **Concept Explanation (Sinhala):** පහත වැරදි කේතය නිවැරදි කර ඇති ආකාරය බලන්න. `Child` එකෙන් `text` කියලා යවද්දී `Grandchild` බලාපොරොත්තු වෙන්නේ `message` එකයි. එය නිවැරදි කර ඇත.
- **Code Block:**

```
// BROKEN CODE
function Parent() {
  return <Child message="Hello" />;
}

function Child({ message }) {
  return <Grandchild text={message} />;
}

function Grandchild({ message }) {
  return <p>{message}</p>;
}

// FIXED VERSION
function Parent() {
  return <Child message="Hello" />;
}

function Child({ message }) {
  return <Grandchild message={message} />;
}

function Grandchild({ message }) {
  return <p>{message}</p>;
}
```

28. Final Beginner Mental Model (අවසාන මානසික ආකෘතිය)

- **Concept Explanation (Sinhala):** React ඉගෙන ගන්නකොට හැමතිස්සෙම මතක තියාගන්න ඕනේ රන්වන් රීතිය තමයි: **"සරලව පටන් ගන්න" (Start Simple).** මුලින්ම හැමදේටම `useState` විතරක් පාවිච්චි කරන්න. අවශ්‍ය වුණොත් විතරක් state එක parent ට `lift` කරන්න. Prop drilling ප්‍රශ්නයක් වෙන මට්ටමටම app එක ලොකු වුණොත් විතරක් `Context API` එකට යන්න. App එකේ state එක ඉතාම සංකීර්ණ වුණොත් විතරක් `Redux Toolkit` හෝ `Zustand` වගේ external library එකක් පාවිච්චි කරන්න. ඒ වගේම, ප්‍රශ්නයක් ආවාම අනුමාන කරන්නේ නැතුව කෙලින්ම `React DevTools` open කරලා ඇස් දෙකෙන්ම බලාගෙන ප්‍රශ්නය විසඳන්න පුරුදු වෙන්න.

```
eof
```